



SDEV 2451

Full Stack Web Development

Global State Management and Authentication - 3

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments
- No Cheating
- Be a good classmate
- Don't waste your time
- Show up to class

Agenda

On the right is what we will cover today.

- JWT Authentication Flow
- SimpleJWT Backend Setup
- Registration and Profile Views
- URL Routing and Endpoint Testing
- Token Storage in the Browser
- API Client with Token Refresh
- Auth API Functions
- AuthContext — Global Auth State
- useAuth Hook
- Wiring App.jsx
- Protected Routes
- Navbar with Logout
- Key Takeaways

JWT Authentication Flow

JWTs allow stateless authentication — the server validates each request with a cryptographic signature, no session storage needed.

The three phases in a SPA:

- **Login** — client sends credentials; server returns `{ access, refresh }` tokens
- **Authenticated request** — client attaches the access token as `Authorization: Bearer <token>` on every API call
- **Silent refresh** — when a 401 is received, the client exchanges the refresh token for a new access token without prompting the user

SimpleJWT Backend Setup

Install `djangorestframework-simplejwt` then configure DRF to validate JWT tokens on every protected request.

```
# settings.py
INSTALLED_APPS = [
    # ... Django built-ins ...
    "rest_framework",
    "rest_framework_simplejwt",
    "corsheaders",
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
}

SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=60),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

SimpleJWT Backend Setup (continued)

- `JWTAuthentication` tells DRF to look for `Authorization: Bearer <token>` and return 401 if it is missing or invalid
- Short `ACCESS_TOKEN_LIFETIME` limits damage from an intercepted token — it expires quickly
- Long `REFRESH_TOKEN_LIFETIME` keeps users logged in without re-entering credentials
- `CorsMiddleware` must be the first entry in `MIDDLEWARE` so the browser's same-origin policy doesn't block React dev server requests

Registration and Profile Views

`AllowAny` on the register view lets unauthenticated users sign up; `IsAuthenticated` on the profile view requires a valid token.

```
# views.py
class UserRegistrationView(APIView):
    permission_classes = (AllowAny,)

    def post(self, request):
        serializer = UserRegistrationSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response({"message": "Registered."}, status=201)
        return Response(serializer.errors, status=400)

class MeView(APIView):
    permission_classes = (IsAuthenticated,)

    def get(self, request):
        user = request.user
        return Response({"id": user.id, "username": user.username, "role": user.role})
```

Registration and Profile Views (continued)

- `UserRegistrationSerializer` calls `create_user()` to hash the password — never call `User.objects.create()` with a plain-text password
- `serializer.errors` is a dict of field-level validation messages: `{"username": ["Already taken."]}`
- `request.user` is populated by `JWTAuthentication` once the token is validated — no manual token parsing needed in the view
- `MeView` returns `role`, which is not encoded in the JWT payload — the frontend calls this endpoint after login to build the user object

URL Routing and Endpoint Testing

Wire the views to URL paths; `TokenObtainPairView` and `TokenRefreshView` are built-in SimpleJWT views — no custom code needed.

```
# core/urls.py
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns = [
    path("register/", UserRegistrationView.as_view()),
    path("login/", TokenObtainPairView.as_view()),
    path("token/refresh/", TokenRefreshView.as_view()),
    path("me/", MeView.as_view()),
]
```

URL Routing and Endpoint Testing (continued)

Method	URL	Description
POST	/api/v1/auth/register/	Create account
POST	/api/v1/auth/login/	Get access + refresh tokens
POST	/api/v1/auth/token/refresh/	Renew access token
GET	/api/v1/auth/me/	Current user profile

- TokenObtainPairView accepts {username, password} and returns {access, refresh} — no custom code
- TokenRefreshView accepts {refresh} and returns a new {access} — the frontend calls this automatically on 401

WE ARE ESSENTIAL TO ALBERTA



Token Storage in the Browser

Centralise all `localStorage` access behind named functions so components never read or write tokens directly.

```
// tokenStorage.js
const ACCESS_KEY = 'app_access'
const REFRESH_KEY = 'app_refresh'
const USER_KEY = 'app_user'

export const getAccessToken = () => localStorage.getItem(ACCESS_KEY)
export const setAccessToken = (t) => localStorage.setItem(ACCESS_KEY, t)
export const getRefreshToken = () => localStorage.getItem(REFRESH_KEY)
export const setRefreshToken = (t) => localStorage.setItem(REFRESH_KEY, t)

export function clearStoredTokens() {
  localStorage.removeItem(ACCESS_KEY)
  localStorage.removeItem(REFRESH_KEY)
  localStorage.removeItem(USER_KEY)
}
```

Token Storage in the Browser (continued)

- Prefixed key names prevent collisions with other apps on the same origin — `app_access` not just `access`
- One file owns all reads and writes — if the storage strategy ever changes (e.g. to `sessionStorage`), only this file needs updating
- `clearStoredTokens()` removes everything atomically — no stale credentials remain after logout
- `localStorage` persists across page refreshes; `sessionStorage` clears on tab close — choose based on session length requirements

API Client with Token Refresh

A thin `fetch` wrapper that attaches `Authorization` automatically and retries silently after a 401.

```
// client.js
let _refreshPromise = null

async function apiClient(endpoint, options = {}) {
  const token = getAccessToken()
  const headers = {
    'Content-Type': 'application/json',
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  }
  let res = await fetch(`${BASE_URL}${endpoint}`, { ...options, headers })

  if (res.status === 401 && _onTokenRefresh && getAccessToken()) {
    if (!_refreshPromise) {
      _refreshPromise = _onTokenRefresh().finally(() => { _refreshPromise = null })
    }
    const newToken = await _refreshPromise
    res = await fetch(`${BASE_URL}${endpoint}`,
      { ...options, headers: { ...headers, Authorization: `Bearer ${newToken}` } })
  }
  return res
}
```

API Client with Token Refresh (continued)

- Token is read at call-time inside the function — always uses the latest stored value, never a stale closure
- `_refreshPromise` deduplicates concurrent 401s: if three requests expire at the same time, only one `POST /token/refresh/` is sent
- Anonymous requests (login, register) reach the server without a token — the `Authorization` header is only added when one exists
- `setAuthCallbacks({ onTokenRefresh, onLogout })` lets `AuthContext` register callbacks without the client having a direct React dependency

Auth API Functions

Four thin functions wrap each auth endpoint — they build the request and delegate to `apiClient`, nothing more.

```
// auth.js
import apiClient from './client'

export async function loginUser({ username, password }) {
  return apiClient('/auth/login/', {
    method: 'POST',
    body: JSON.stringify({ username, password }),
  })
}

export async function refreshToken({ refresh }) {
  return apiClient('/auth/token/refresh/', {
    method: 'POST',
    body: JSON.stringify({ refresh }),
  })
}

export async function fetchMe() {
  return apiClient('/auth/me/')
}
```

Auth API Functions (continued)

- Each function has one job: build the request and return the raw `Response` — no error handling, no state changes
- Returning the raw `Response` gives callers full control: they check `res.ok`, parse the body, and decide how to surface errors
- `fetchMe()` is called right after login to get `username` and `role` — fields that are not encoded in the JWT payload itself
- The separation means the API layer can be tested independently of any React component

AuthContext — Global Auth State

`AuthContext` is the single source of truth for the logged-in user and access token — any component in the tree can read or update auth state via context.

```
// AuthContext.jsx
export const AuthContext = createContext(null)

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => getStoredUser())
  const [token, setToken] = useState(() => getAccessToken())

  useEffect(() => {
    setAuthCallbacks({
      onTokenRefresh: async () => {
        const res = await refreshToken({ refresh: getRefreshToken() })
        const data = await res.json()
        setAccessToken(data.access); setToken(data.access)
        return data.access
      },
      onLogout: () => { clearAuthState(); window.location.href = '/login' },
    })
  }, [])
  // loginMutation, registerMutation, logout ...
```



AuthContext — Global Auth State (continued)

- Lazy `useState(() ⇒ getStoredUser())` reads from `localStorage` once on mount — restores the session instantly on page refresh
- `useEffect( ... , [])` registers refresh/logout callbacks with `apiClient` once after mount — defined inside the effect so they close over the latest state setters
- `window.location.href = '/login'` in `onLogout` is a hard redirect — used because `onLogout` fires inside `apiClient`, outside the React component tree
- `loginMutation.mutationFn` calls `setAccessToken()` before `fetchMe()` — the `/me/` request needs the token in `localStorage` when it fires

useAuth Hook

A custom hook wraps `useContext(AuthContext)` and adds a guard so misconfigured trees fail with a clear error.

```
// useAuth.js
import { useContext } from 'react'
import { AuthContext } from '../contexts/AuthContext'

export function useAuth() {
  const context = useContext(AuthContext)
  if (!context) {
    throw new Error('useAuth must be used inside an AuthProvider')
  }
  return context
}
```

useAuth Hook (continued)

- Components call `useAuth()` and never import `AuthContext` or `useContext` directly — one import point for all auth state
- The `if (!context)` guard converts the `null` default into a descriptive error — without it, destructuring `null` produces a confusing `TypeError` with no hint about the missing provider
- If the context shape changes, only `useAuth.js` and `AuthContext.jsx` need updating — all consumers stay the same

Wiring App.jsx

`AuthProvider` must be inside `QueryClientProvider` (uses `useMutation`) and outside `BrowserRouter` (logout uses `window.location.href`).

```
// App.jsx
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <BrowserRouter>
          <Navbar />
          <Routes>
            <Route path="/" element={<HomePage />} />
            <Route path="/login" element={<LoginPage />} />
            <Route path="/dashboard" element={
              <ProtectedRoute><DashboardPage /></ProtectedRoute>
            } />
          </Routes>
        </BrowserRouter>
      </AuthProvider>
    </QueryClientProvider>
  )
}
```

Wiring App.jsx (continued)

- `AuthProvider` inside `QueryClientProvider` — `useMutation` requires a `QueryClient` in the tree above it
- `AuthProvider` outside `BrowserRouter` — the silent logout uses `window.location.href`, not `useNavigate`, so it doesn't need Router context
- `Navbar` is a separate component so it can call `useAuth()` — `App` renders the providers, so context isn't available at that level
- `LoginPage` passes a per-call `onSuccess` callback to `login()` for navigation — context's `onSuccess` handles token storage; the page handles routing

Protected Routes

`ProtectedRoute` reads `user` from context and redirects unauthenticated visitors before the protected page renders.

```
// ProtectedRoute.jsx
import { Navigate } from 'react-router-dom'
import { useAuth } from '../hooks/useAuth'

function ProtectedRoute({ children }) {
  const { user } = useAuth()

  if (!user) {
    return <Navigate to="/login" replace />
  }

  return children
}

// Usage in App.jsx:
<Route path="/dashboard" element={
  <ProtectedRoute><DashboardPage /></ProtectedRoute>
} />
```

Protected Routes (continued)

- `user` is `null` before login — the same initial value `AuthContext` sets — so the check is a simple falsy test
- `replace` on `<Navigate>` replaces the current history entry — pressing Back after being redirected to login won't return the user to the protected page
- Adding protection to a new route is a one-line change in `App.jsx` — wrap the element with `<ProtectedRoute>`
- Public pages don't need `ProtectedRoute` — only pages that require an account to access

Navbar with Logout

Extract the navbar into its own component so it can call `useAuth()` and switch between logged-in and logged-out views.

```
// Navbar.jsx
function Navbar() {
  const { user, logout } = useAuth()
  const navigate = useNavigate()

  function handleLogout() {
    logout() // clears React state + localStorage
    navigate('/login')
  }

  return (
    <nav>
      {user ? (
        <span>Hi, {user.username}</span>
        <button onClick={handleLogout}>Log Out</button>
      ) : (
        <NavLink to="/login">Login</NavLink>
        <NavLink to="/register">Register</NavLink>
      )}
    </nav>
  )
}
```



Navbar with Logout (continued)

- `{user ? ( ... ) : ( ... )}` re-renders automatically whenever auth state changes — no extra event wiring needed
- `logout()` is called before `navigate('/login')` — clear state first so a protected route can't momentarily read the old user during the redirect
- `useNavigate` is available here because `Navbar` renders inside `BrowserRouter` — prefer it over `window.location.href` when inside the component tree
- Extracting the navbar was necessary: `App` renders the providers, so hooks like `useAuth()` can't be called there

Key Takeaways

- **JWT flow** — login returns short-lived access + long-lived refresh tokens; the client attaches the access token on every request and silently renews it on 401
- **localStorage persistence** — lazy `useState` initialisers read stored credentials on mount, restoring sessions instantly across page refreshes
- **React Context for global auth** — `AuthContext` is the single source of truth for `user` and `accessToken`; components access it via `useAuth()`
- **Separation of concerns** — `tokenStorage.js` owns storage, `client.js` owns HTTP transport, `AuthContext` owns React state — each layer has one responsibility and can change independently
- **Silent refresh deduplication** — `_refreshPromise` ensures multiple concurrent 401s trigger only one refresh call to the backend

WE ARE ESSENTIAL
TO ALBERTA





Example

Let's go do an example together

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS